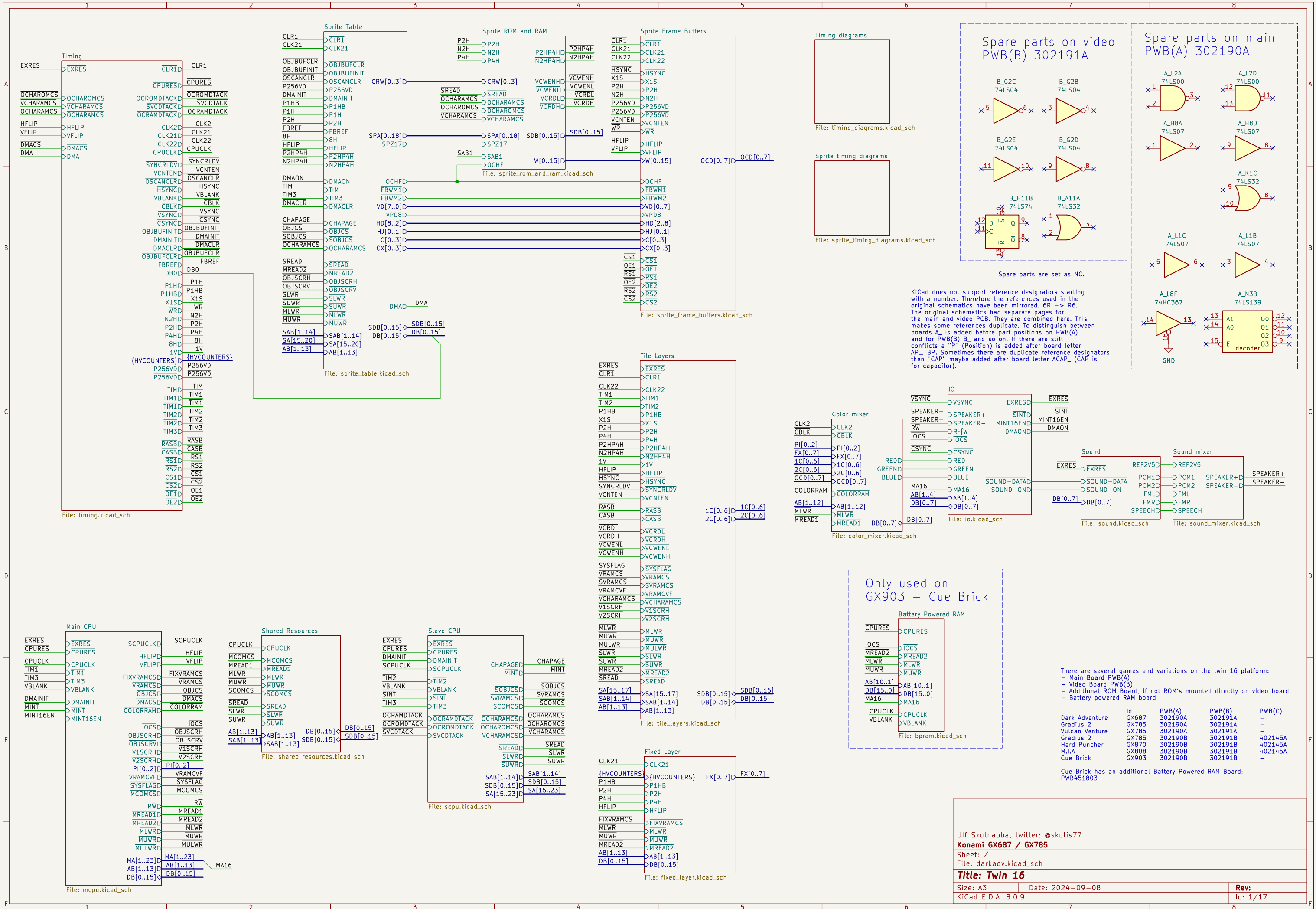
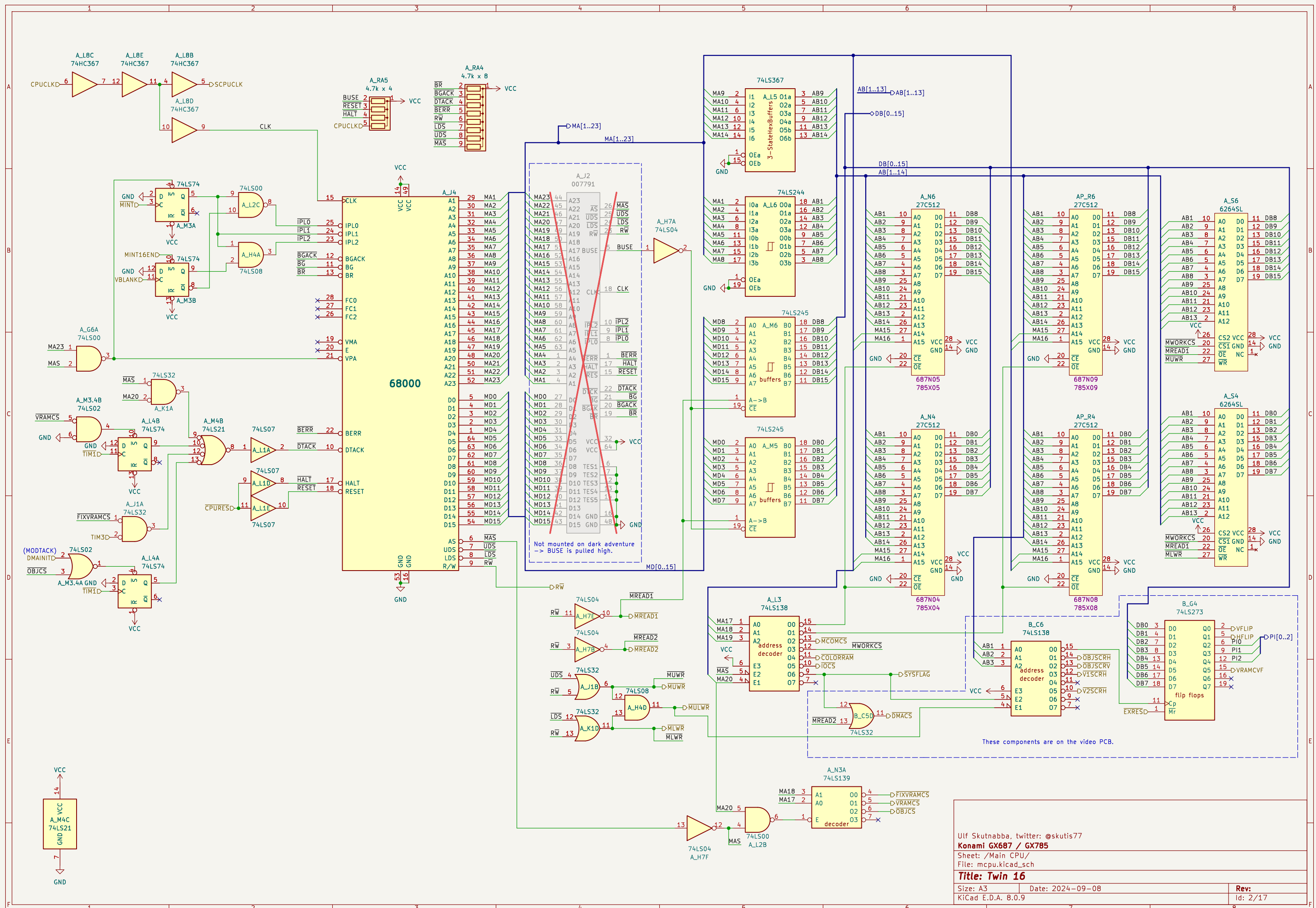
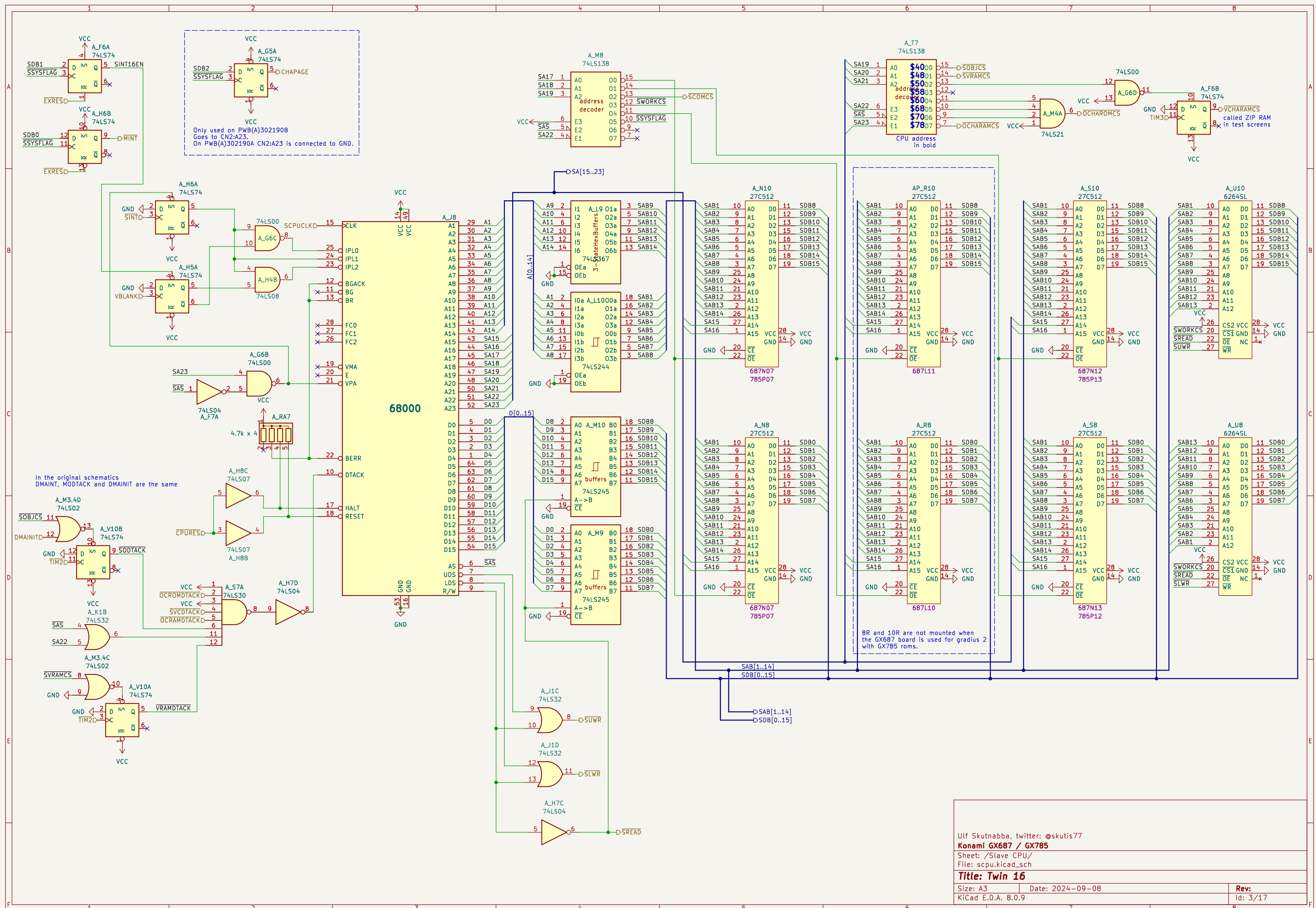




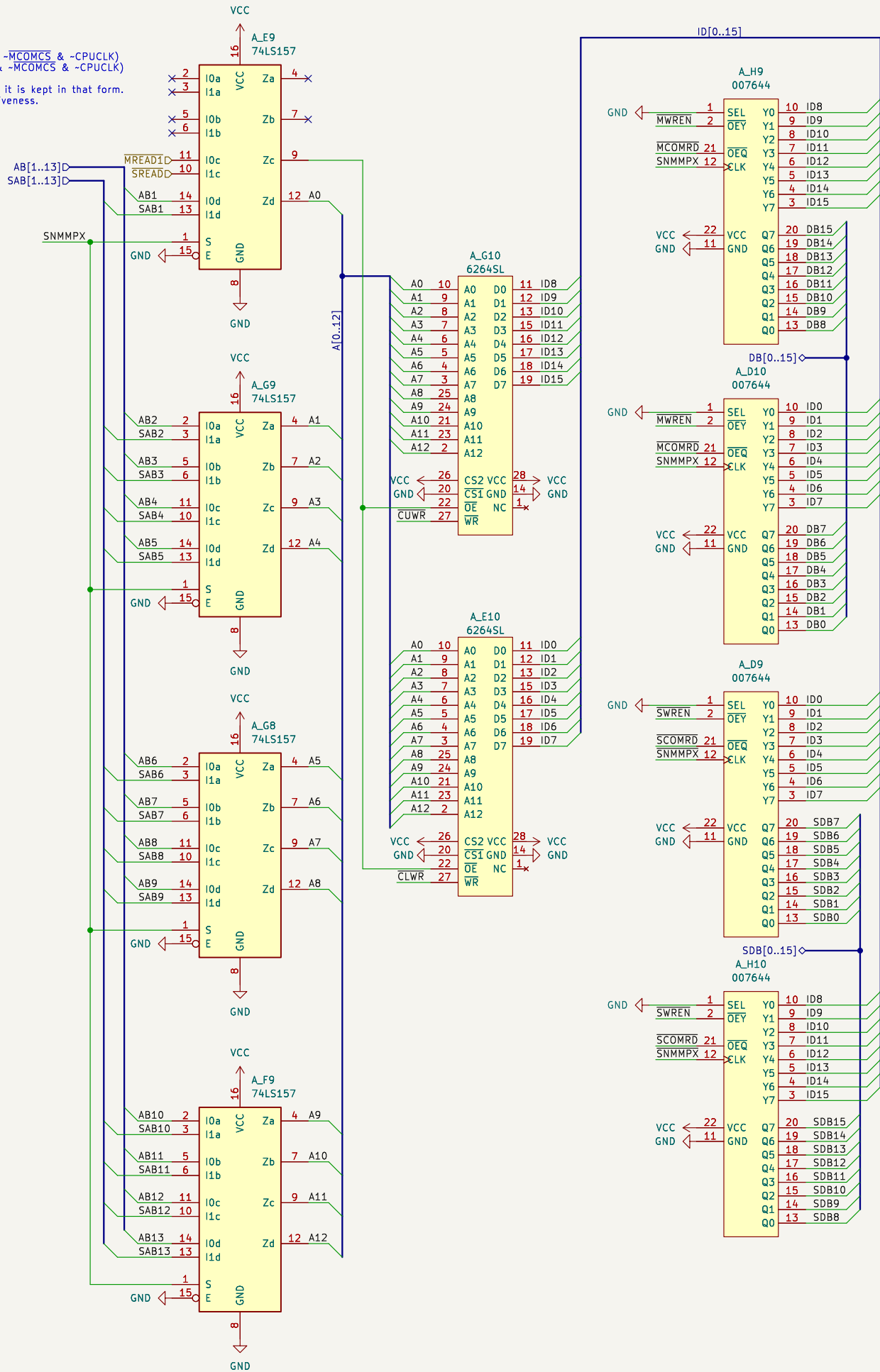
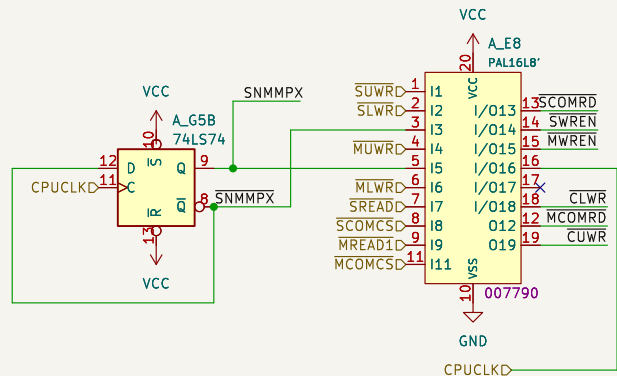




Konami 007790

$\overline{MCOMRD} = \neg(\neg \overline{MREAD1} \& \neg \overline{MCOMCS}) = \overline{MREAD1} \mid \overline{MCOMCS}$
 $\overline{SCOMRD} = \neg(\neg \overline{SREAD} \& \neg \overline{SCOMCS})$
 $\overline{SWREN} = \neg(\neg \overline{SNMMPX} \& \overline{SREAD} \& \neg \overline{SCOMCS})$
 $\overline{MWREN} = \neg(\neg \overline{SNMMPX} \& \overline{MREAD1} \& \neg \overline{MCOMCS})$
 $\overline{CLWR} = \neg(\neg \overline{SLWR} \& \neg \overline{SNMMPX} \& \overline{SNMMPX} \& \overline{SREAD} \& \neg \overline{SCOMCS} \& \neg \overline{CPUCLK} \mid \neg \overline{SNMMPX} \& \neg \overline{MLWR} \& \overline{MREAD1} \& \neg \overline{MCOMCS} \& \neg \overline{CPUCLK})$
 $\overline{CUWR} = \neg(\neg \overline{SUWR} \& \neg \overline{SNMMPX} \& \overline{SNMMPX} \& \overline{SREAD} \& \neg \overline{SCOMCS} \& \neg \overline{CPUCLK} \mid \neg \overline{MUWR} \& \neg \overline{SNMMPX} \& \overline{MREAD1} \& \neg \overline{MCOMCS} \& \neg \overline{CPUCLK})$

All AND logic can be converted to OR form. But since AND logic and inverters are used in the PAL16L8 device it is kept in that form. The equations follow verilog syntax. The active low signals are not inverted, the bar is there only to show activeness.



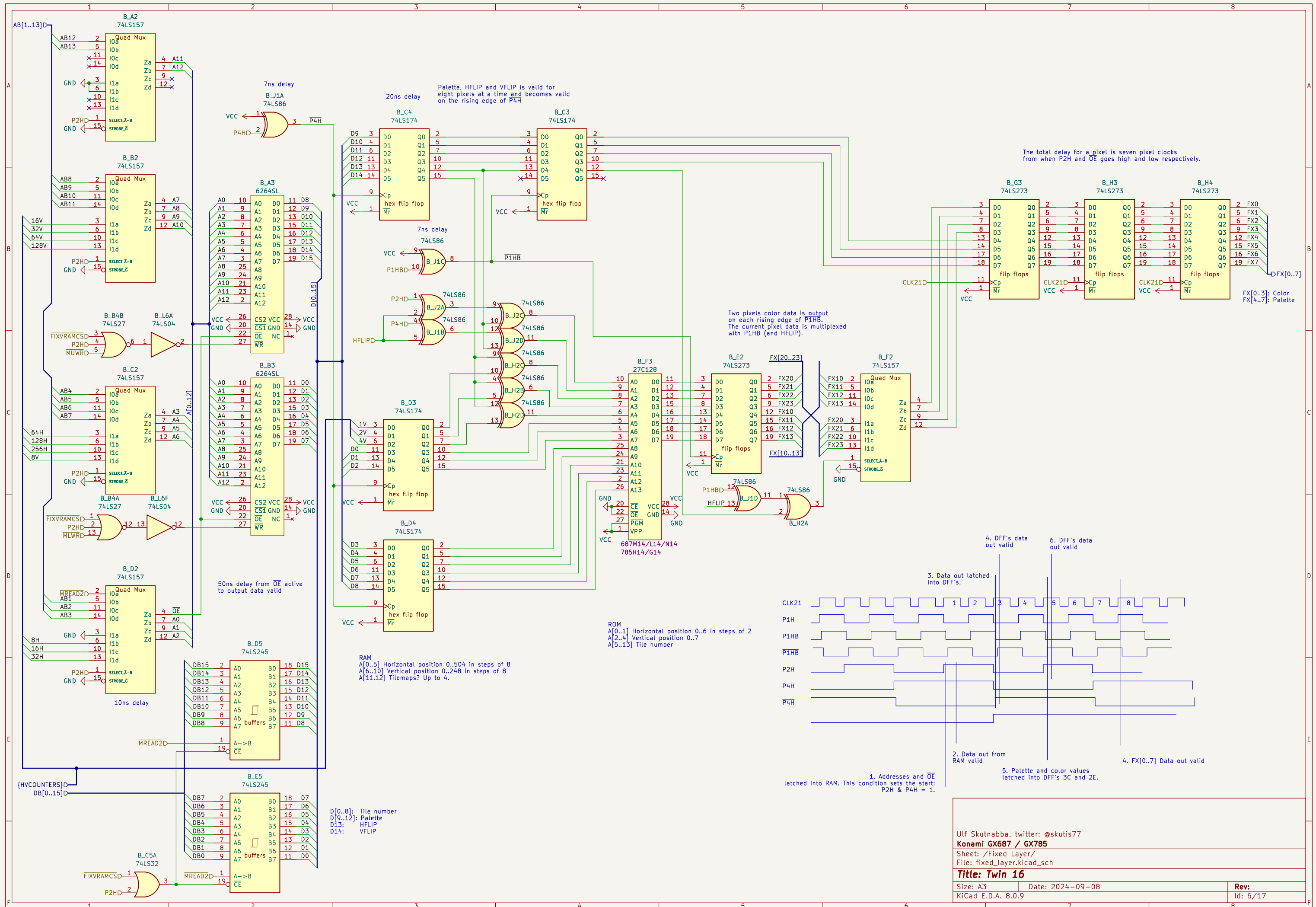
This is from jammarcade.net

Pin 1 = SEL – changes whether the Q outputs require a rising edge clock or not.
Pin 2 = OEY – Output enable for the Y outputs else they are inputs. Active LOW
Pin 21 = OEQ – Output enable for the Q (clocked) outputs else they are inputs. Active LOW
Pin 12 = CLK – Clock input

Pins 3–10 = Y outputs – normal outputs. If OEY is LOW these pins become outputs mirroring the state of the Q input pins. SEL and CLK pins are not used in this mode.

Pins 13–20 = Q outputs – clocked outputs. If OEQ is LOW these pins become outputs. If SEL is HIGH and CLK is HIGH these outputs will mirror the state of the Y inputs. If SEL is LOW then the outputs will only change state on a rising edge CLK.





Ulf Skutnabba, twitter: @skutis77

Konami GX687 / GX785

Sheet: /Fixed Layer/

File: fixed_layer.kicad_sch

Title: Twin 16

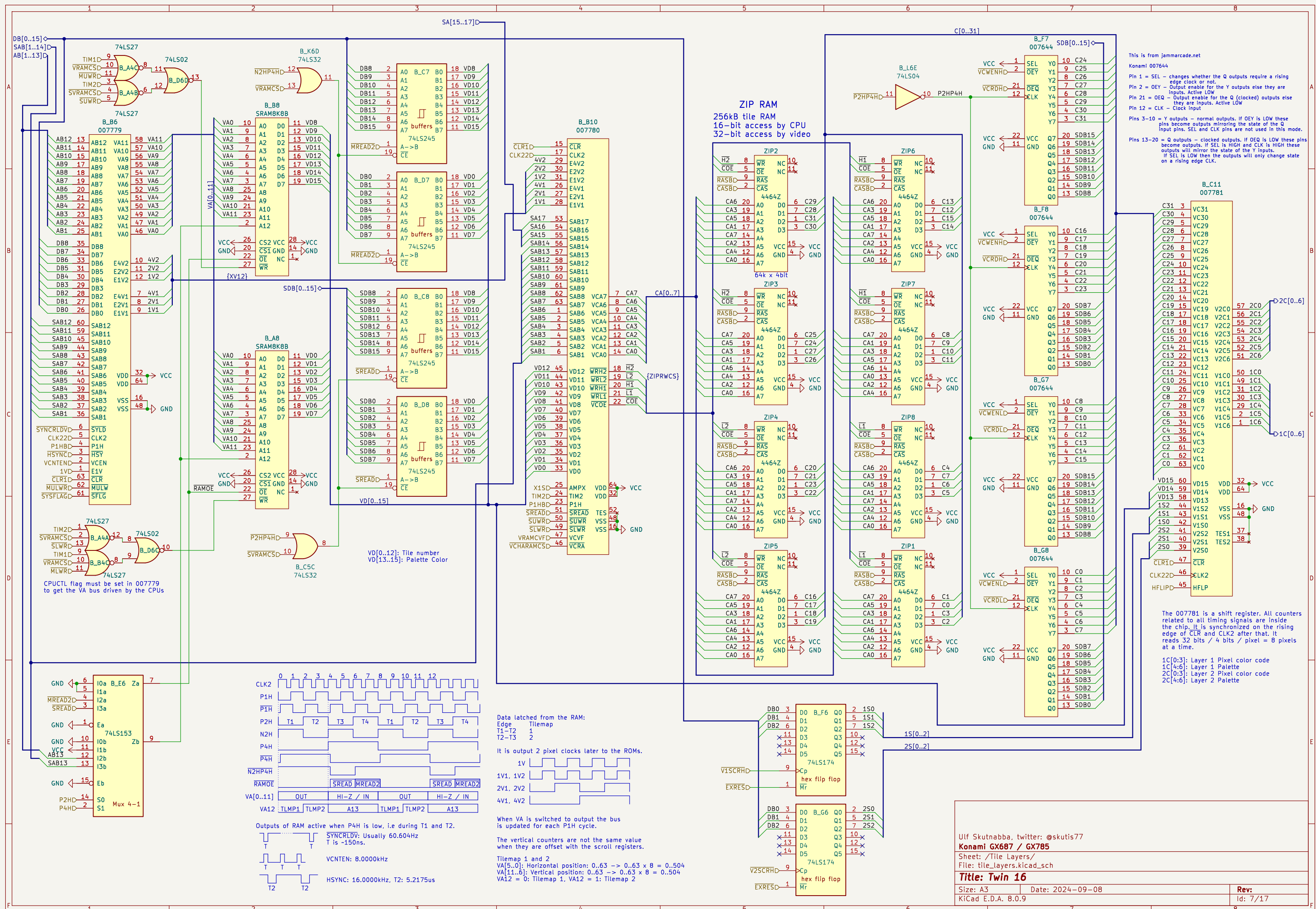
Size: A3

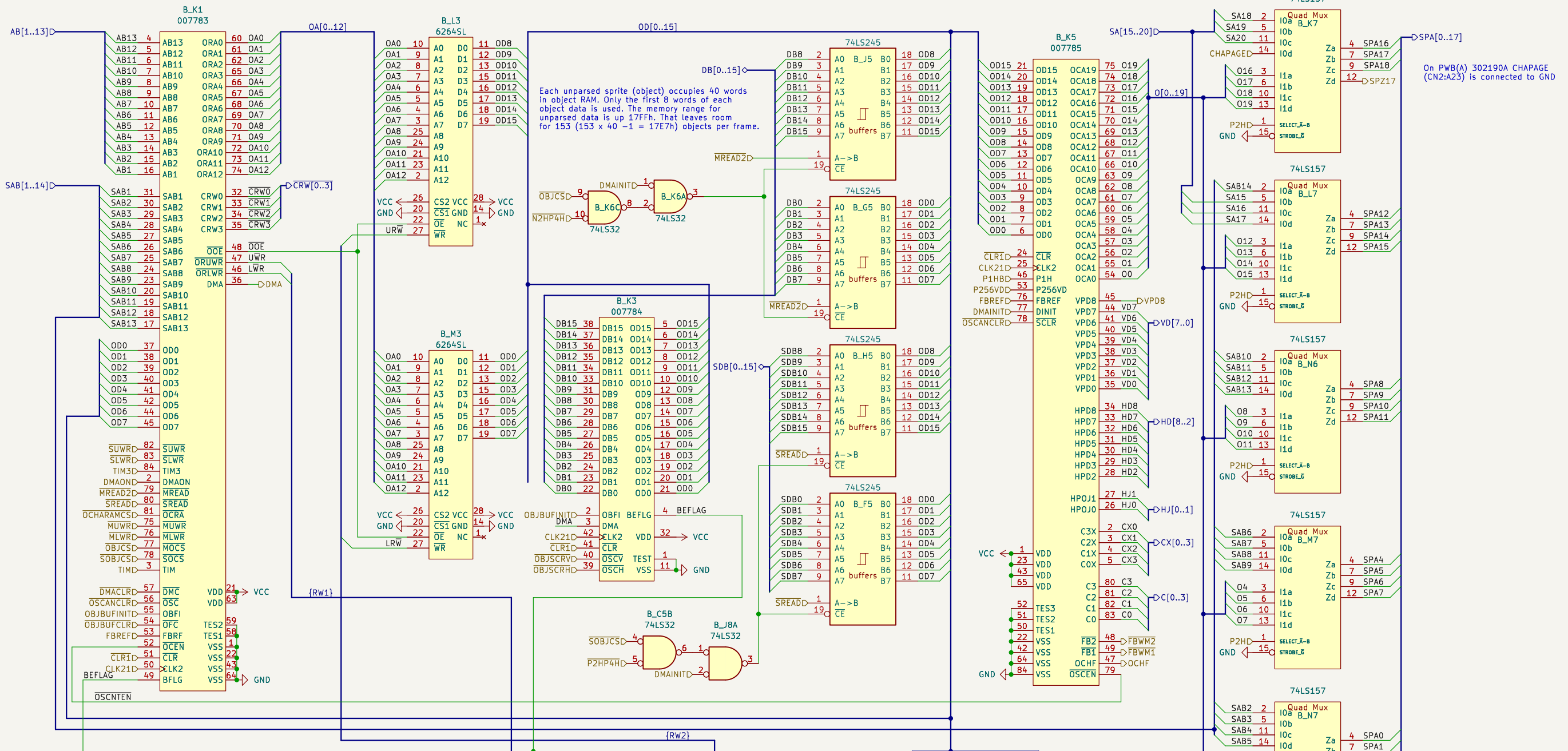
Date: 2024-09-08

Rev:

KiCad E.D.A. 8.0.9

Id: 6/17





007784 Sprite parser

- subtracts the scroll value for the sprites from current coordinates for the sprites.
- When DMA goes high. The 007784 will read 8 words, if bit OD15 of the first word is high then BEFLAG goes high and stays high until the sprite RAM has processed the first four words and possibly updated the four following words at the high end of the RAM. It will compress the eight words into four words and also subtract the x- and y-scroll values from the x and y coordinates. It will write the four words at the high end of the sprite RAM.
- Addresses at the low end goes from 0x0000..0x0007 and the following cycle at 0x0028..0x002F..

Scroll Register
 OBJSCRV Sprite horizontal scroll
 OBJSCRH Sprite vertical scroll

Data is latched in the scroll registers when the signals OBJSCRV and OBJSCRH go low.

007793

CLK21' = CLK21

LRW = -(DMAONS & OBJBUFINIT & CLK21' | BEFLAG & DMA & P1H & DMAONS & CLK21' & (-BH & HFLIP | 8H & -HFLIP) -LRW & -DMAINIT)

URW = -(DMAONS & OBJBUFINIT & CLK21' | BEFLAG & DMA & P1H & DMAONS & CLK21' & (BH & -HFLIP | -BH & HFLIP) -URW & -DMAINIT)

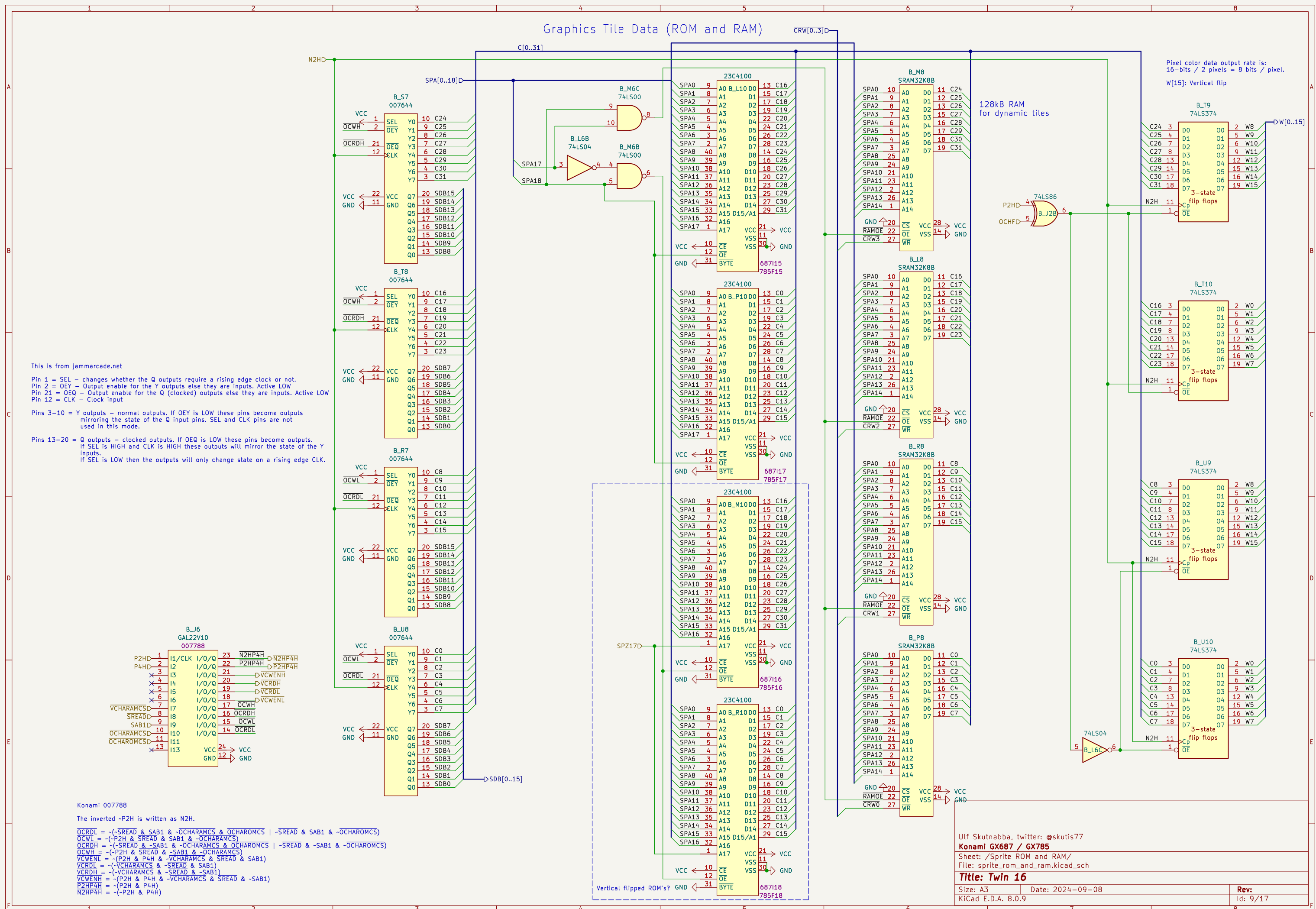
The sprite memory is controlled in three separate cases:

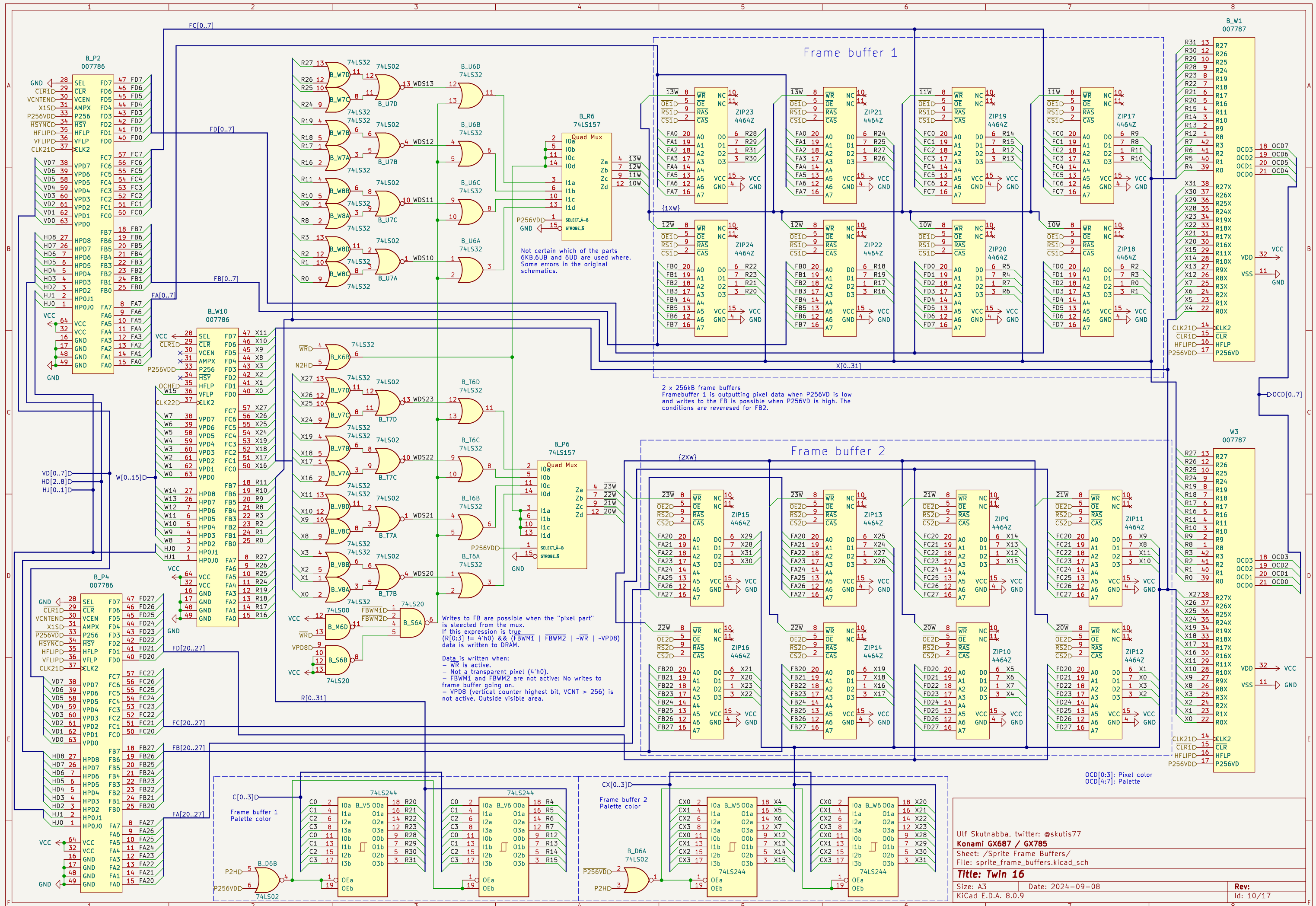
1. OBJBUFINIT
2. BEFLAG & DMA
3. -DMAINIT

Ulf Skutnabba, twitter: @skutis77
Konami GX687 / GX785
 Sheet: /Sprite Table/
 File: sprite_table.kicad_sch

Title: Twin 16

Size: A3	Date: 2024-09-08	Rev:
KiCad E.D.A. 8.0.9		Id: 8/17



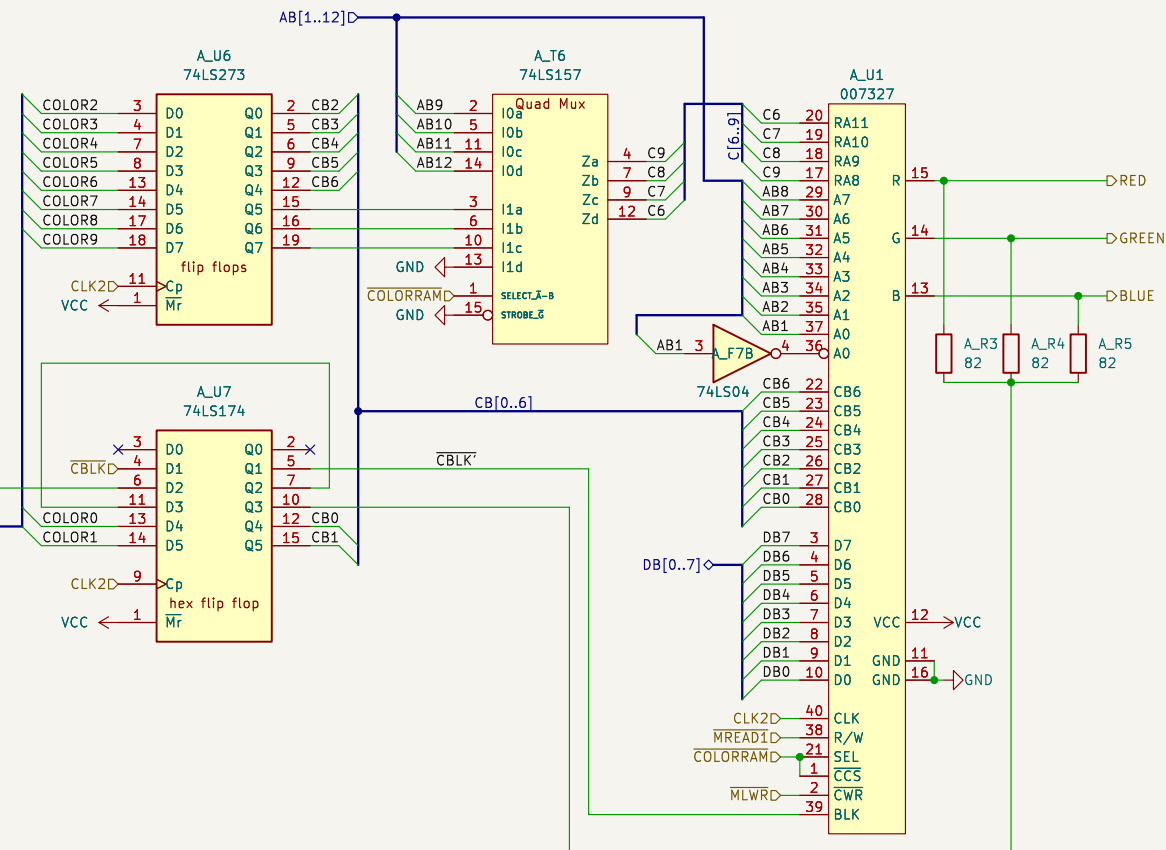
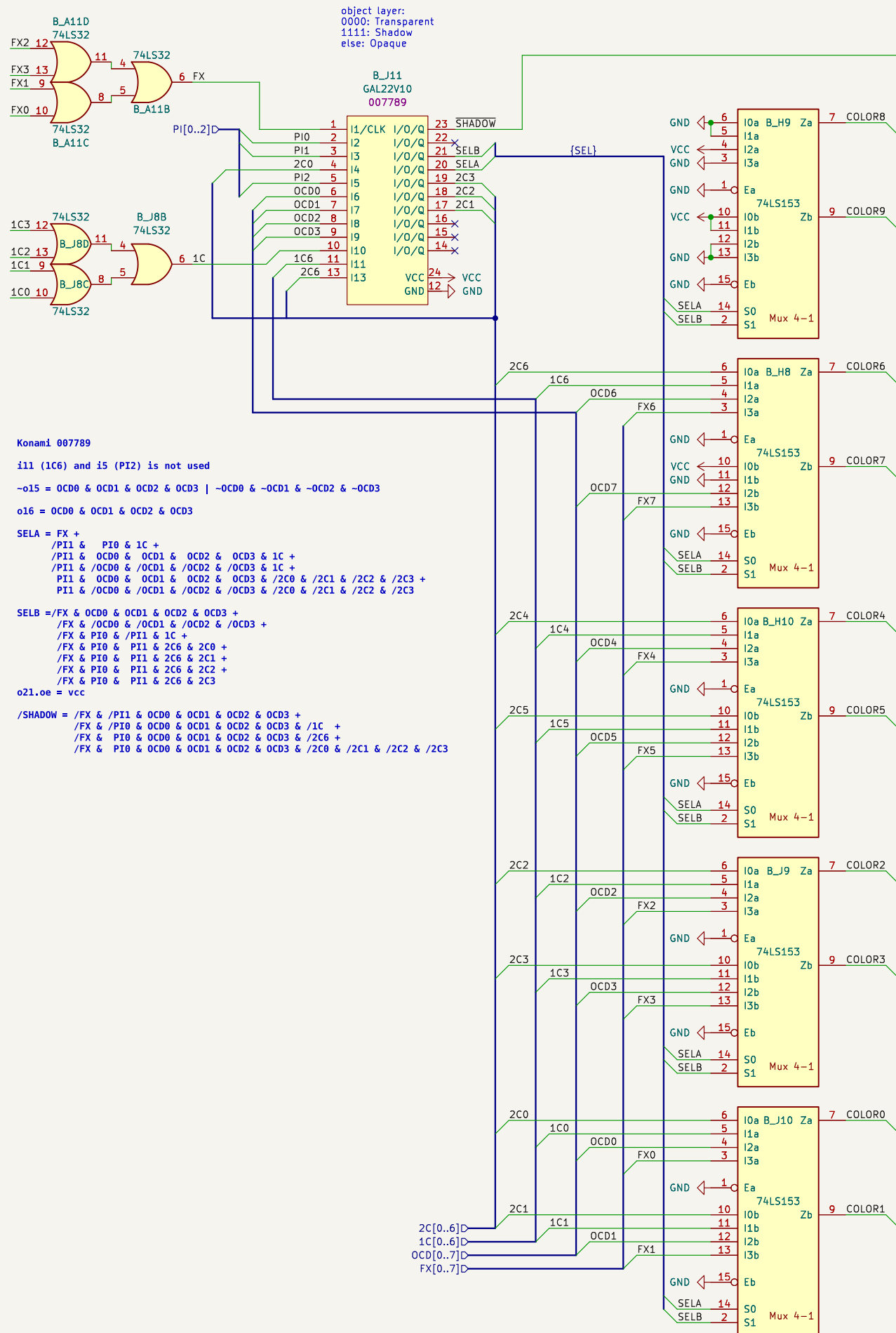


Ulf Skutnabba, twitter: @skutis77
Konami GX687 / GX785
Sheet: /Sprite Frame Buffers/
File: sprite_frame_buffers.kicad_sch
Title: Twin 16
Size: A3 Date: 2024-09-08
KiCad E.D.A. 8.0.9

Rev:
Id: 10/17

Components are on Video PCB

Components are on Main PCB



Ulf Skutnabba, twitter: @skutis77

Konami GX687 / GX785

Sheet: /Color mixer/

File: color_mixer.kicad_sch

Title: Twin 16

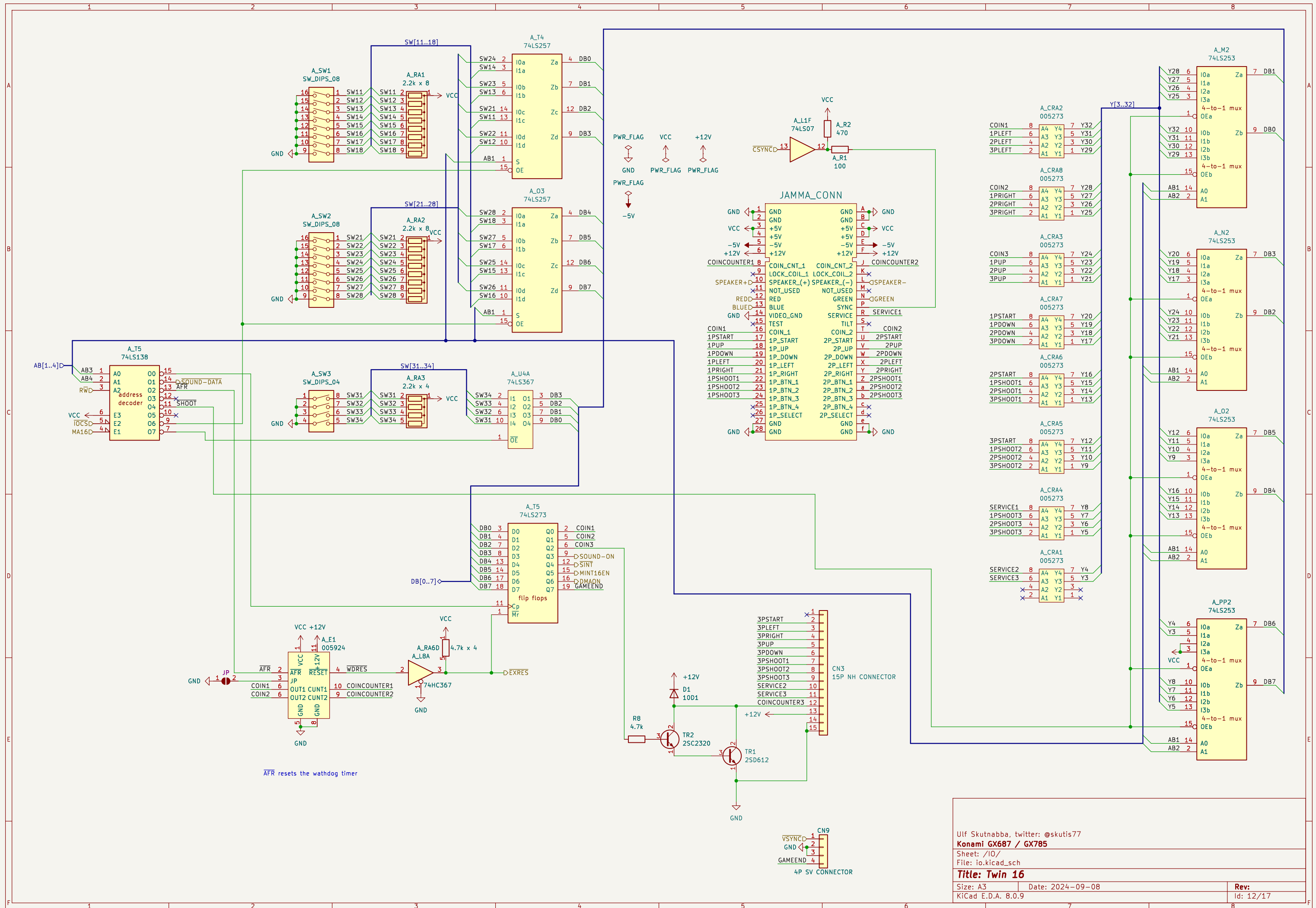
Size: A3

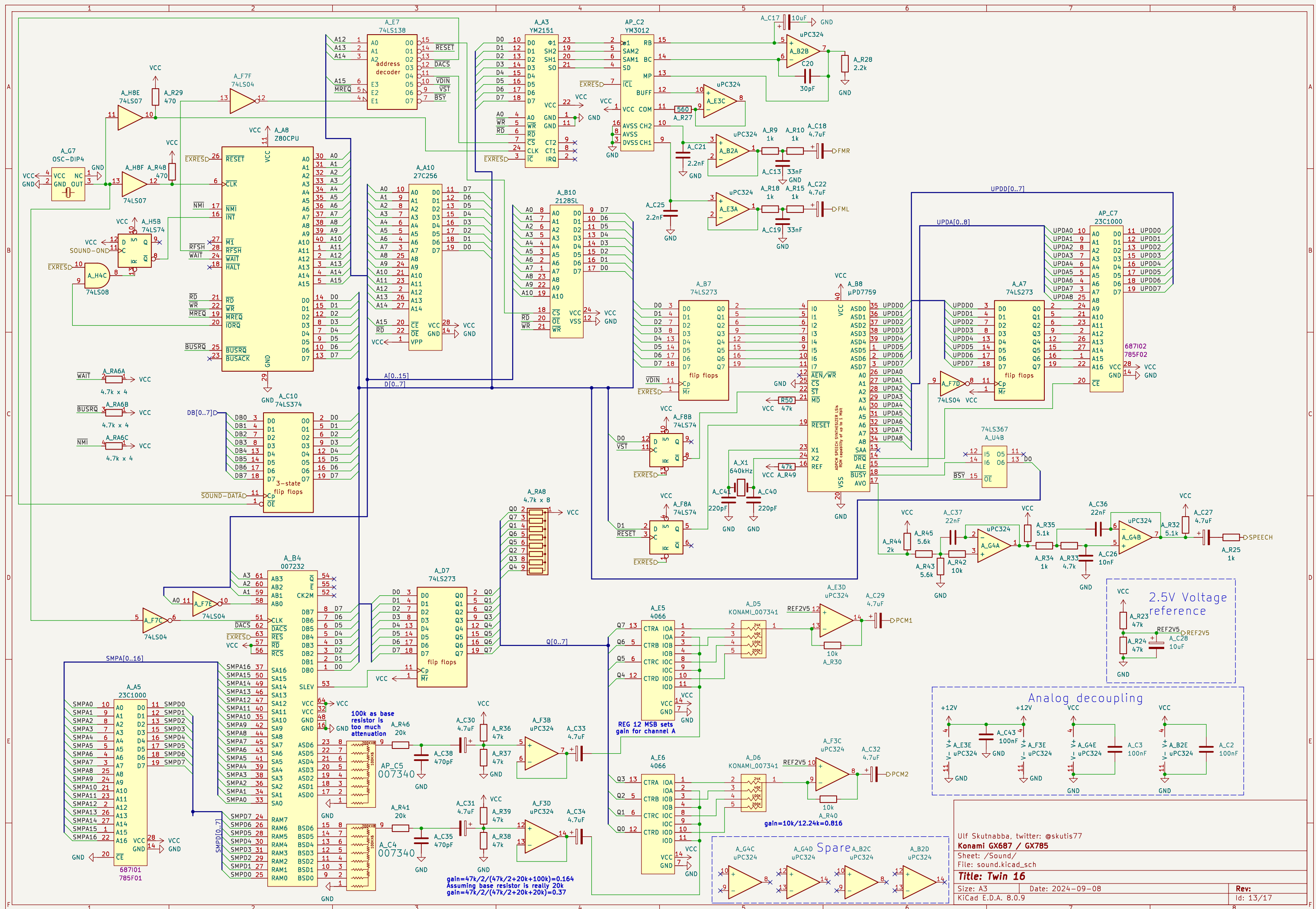
Date: 2024-09-08

Rev:

KiCad E.D.A. 8.0.9

Id: 11/17





Timing diagram for the HPS/PS7 clock system. The diagram shows multiple clock signals over time. The signals are: CLR (clear), P4H (768kHz), 8H (384kHz), 16H (192kHz), 32H (96kHz), 64H (48kHz), 128H (32kHz), 256H (16kHz), HSYNC (16kHz), HBLK (8kHz), CSYNC (8kHz), VCNTEN (8kHz), and 1V (power supply). The diagram illustrates the relationship between these signals, showing how the HPS/PS7 clock is derived from the 16kHz HSYNC signal and how the 8kHz HBLK signal is derived from the 16kHz HSYNC signal. The 1V signal is shown as a step function, indicating the power supply state.

The first HSYNC is at HCOUNT 175 after reset.

$f = 16\text{kHz} / 2 = 8\text{kHz}$ 1V

$f = 16\text{kHz} / 4 = 4\text{kHz}$ 2V

$f = 16\text{kHz} / 8 = 2\text{kHz}$ 4V



The numbers in the HSYNC and HBLK diagrams are HSYNC cycles.
All edges are synchronised to the rising edge of CLK2.

Pixel clock CLK2 = $18.432\text{MHz} / 3 = 6.144\text{MHz}$

Horizontal lines
HSYNC freq = $6.144\text{MHz} / 384 = 16\text{kHz}$
HSYNC period = $1 / 16\text{kHz} = 62.5\mu\text{s}$
Pixels / line: 384
Active pixels: 320
Blank pixels: 64

Vertical lines
VSYNC freq = $16\text{kHz} / 264 = 60.606060\text{Hz}$
VSYNC period = $1 / f = 264 / 16\text{kHz} = 16.5\text{ms}$
scanlines / frame: 264
Active lines: 224
Blank lines: 40

The numbers in the VSYNC and VBLK diagrams are HSYNC cycles.
All edges are synchronised to the falling edge of HSYNC.

Front Porch (FP) 24 px.
Back Porch (BP) 8 px

Front Porch (FP) and
Back Porch (BP) 16 HSYNC cycles.

HCOUNT is bits [256H, 128H, 64H, 32H, 16H, 8H, P4H, P2H, P1H]
VCOUNT is bits [128V, 64V, 32V, 16V, 8V, 4V, 2V, 1V]

CBLK'' is at the output of color mixer 007327.
CBLK' is shifted inside the 007327 one pixel clock.

Rev: 15/17

